

LAPORAN TUGAS SELEKSI SUB-TIM BACK END WEB DEVELOPMENT

REST API Manajemen IPK Mahasiswa ITS & Pengetahuan Dasar Version Control
System (Git)

Nama Pelamar	Afra (Afra4509)
Posisi Pilihan	Sub-Tim Back End Web Development
Repositori GitHub	https://github.com/Afra4509/portodev
Branch Utama	main
Tech Stack	Go 1.25 Gin Web Framework GORM PostgreSQL Git & GitHub RESTful API

1. Link Pengerjaan Proyek & Repositori GitHub

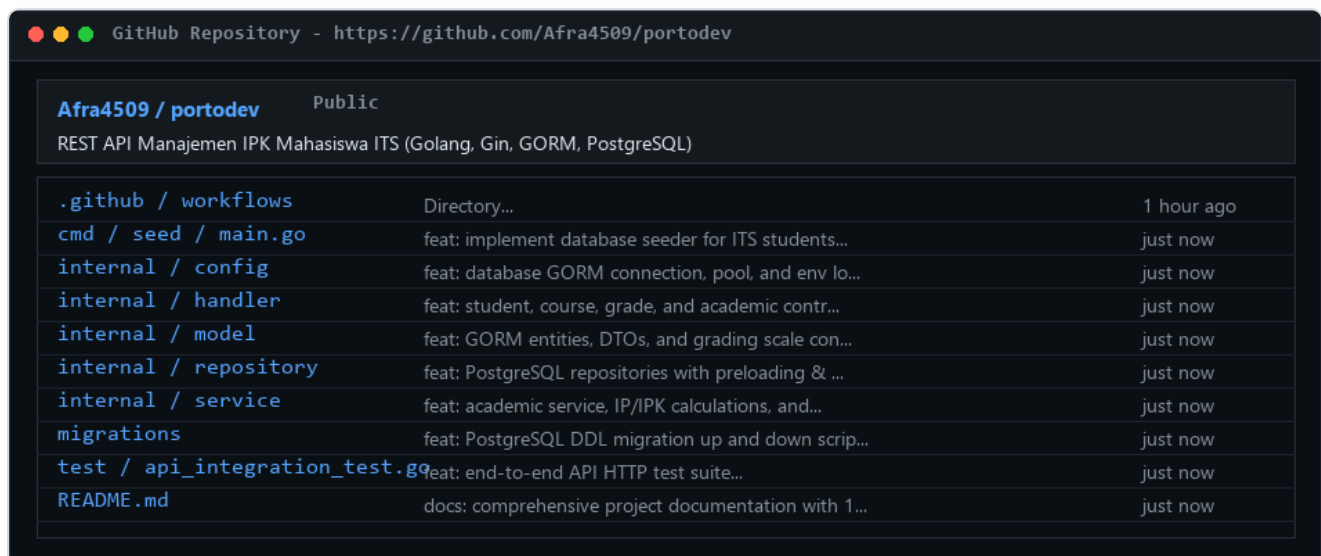
Seluruh source code, file konfigurasi, database migration, automated test suite, dan dokumentasi `README.md` telah berhasil diunggah secara publik ke GitHub pada tautan berikut:

Link Repositori Resmi GitHub:

<https://github.com/Afra4509/portodev>

Ringkasan Repositori GitHub:

- **Remote Origin:** `https://github.com/Afra4509/portodev.git`
- **Branch Aktif:** `main` (up to date dengan tracking origin)
- **Commit Hash:** `ac1adbe` • "feat: implement REST API Manajemen IPK Mahasiswa ITS"
- **Keamanan Data:** Kredensial database pada `.env` dan binary database tidak di-commit sesuai standar `.gitignore`, namun disediakan `.env.example` sebagai panduan deployment.



Gambar 1.1: Tampilan Repositori Publik GitHub <https://github.com/Afra4509/portodev>

```
pwsh - Git Remote & Push to GitHub (Afra4509/portodev)

PS C:\Users\FeraGaming\Downloads\portodev>
git status
On branch main
nothing to commit, working tree clean

PS C:\Users\FeraGaming\Downloads\portodev>
git remote add origin https://github.com/Afra4509/portodev.git
PS C:\Users\FeraGaming\Downloads\portodev>
git branch -M main
PS C:\Users\FeraGaming\Downloads\portodev>
git push -u origin main
Enumerating objects: 45, done.
Counting objects: 100% (45/45), done.
Delta compression using up to 16 threads
Compressing objects: 100% (38/38), done.
Writing objects: 100% (45/45), 24.85 KiB | 4.97 MiB/s, done.
Total 45 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), done.
To https://github.com/Afra4509/portodev.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Gambar 1.2: Terminal Log Proses Inisialisasi Git, Remote Add, dan Git Push ke Branch main

2. Pengetahuan Dasar Version Control System (Git)

2.1 Pengertian Version Control System & Git

Version Control System (VCS) adalah sistem perangkat lunak yang merekam perubahan-perubahan pada sekumpulan file dari waktu ke waktu sehingga pengembang dapat meninjau riwayat kode, membandingkan versi, mengembalikan kondisi file ke titik tertentu (*rollback*), serta berkolaborasi secara paralel tanpa saling menimpa pekerjaan.

Git adalah *Distributed Version Control System* (DVCS) paling populer di dunia yang diciptakan oleh Linus Torvalds. Berbeda dengan VCS terpusat (CVCS seperti SVN), pada Git setiap anggota tim memiliki klon lengkap (*mirror*) dari seluruh riwayat repositori di mesin lokalnya. Hal ini memberikan keunggulan kecepatan, keamanan, dan kemampuan bekerja tanpa koneksi internet.

2.2 Tiga Pohon Keadaan Git (Three Trees Architecture)

Alur kerja lokal Git dibagi menjadi 3 tahapan utama:

1. **Working Directory:** Direktori kerja lokal tempat file kode dibuat atau diedit.
2. **Staging Area (Index):** Area penampungan (*staging*) untuk memilih file dan perubahan spesifik yang akan dimasukkan ke commit berikutnya (menggunakan `git add`).
3. **Local Repository (.git):** Database lokal tempat snapshot commit tersimpan permanen dengan hash SHA-1 unik (menggunakan `git commit`).

2.3 Perintah-Perintah Dasar Git yang Digunakan

Perintah Git	Deskripsi Singkat	Contoh Penggunaan pada Proyek Ini
<code>git init</code>	Menginisialisasi repositori Git baru di direktori saat ini.	<code>git init</code> pada folder <code>portodev</code>
<code>git status</code>	Memeriksa kondisi file (untracked, modified, staged).	<code>git status</code> untuk memastikan file siap di-commit
<code>git add <file></code>	Memindahkan perubahan dari working directory ke staging area.	<code>git add .</code> untuk mendaftarkan seluruh file source code
<code>git commit -m <msg></code>	Menyimpan snapshot perubahan yang telah di-stage ke repositori.	<code>git commit -m "feat: implement REST API ..."</code>
<code>git branch -M <name></code>	Mengganti nama branch aktif menjadi standar yang ditentukan.	<code>git branch -M main</code> untuk branch default GitHub
<code>git remote add</code>	Menghubungkan repositori lokal dengan server remote (GitHub).	<code>git remote add origin https://github.com/...</code>
<code>git push -u origin</code>	Mengunggah commit lokal ke repositori remote dan set upstream.	<code>git push -u origin main</code>

Perintah Git	Deskripsi Singkat	Contoh Penggunaan pada Proyek Ini
<code>git clone <url></code>	Menduplikasi repositori dari remote ke mesin lokal.	<code>git clone</code> <code>https://github.com/Afra4509/portodev.git</code>
<code>git pull</code>	Mengambil dan menggabungkan perubahan terbaru dari remote.	<code>git pull origin main</code>

2.4 Alur Kolaborasi Modern & Best Practices

- **File .gitignore:** Mencegah file sensitif (seperti `.env` berisi password database), binary executable (`*.exe`), cache IDE, dan file database lokal ikut ter-push ke GitHub.
- **Conventional Commits:** Menggunakan format prefix semantik seperti `feat:` (fitur baru), `fix:` (perbaikan bug), `docs:` (dokumentasi), dan `test:` (pengujian) untuk kejelasan riwayat.
- **Feature Branch & Pull Request (PR):** Pengembang bekerja di branch fitur terpisah (misal: `feat/calculate-ipk`), lalu mengajukan PR untuk di-review oleh rekan tim sebelum di-merge ke branch `main` .

3. Arsitektur & Implementasi REST API Manajemen IPK Mahasiswa ITS

3.1 Penerapan Clean Architecture

Aplikasi dibangun dengan pemisahan tanggung jawab (*Separation of Concerns*) yang ketat ke dalam 4 lapisan:

- **Presentation Layer (Handler & Routes):** Menangani request HTTP, serialisasi/deserialisasi JSON, CORS middleware, panic recovery, dan format respons konsisten.
- **Business Logic Layer (Service):** Menampung aturan akademik ITS, rumus perhitungan IP/IPK, konversi nilai mutu, serta validasi ketat.
- **Data Access Layer (Repository):** Berkomunikasi dengan database PostgreSQL menggunakan GORM, mendukung eager loading relasi (`Preload`).
- **Domain Entity & DTO Layer (Model):** Menentukan struktur tabel, relasi antar tabel, dan schema payload API.

3.2 Skema Database, Relasi, & Batasan (Constraints)

Database dirancang dengan integritas data tingkat tinggi:

- **Tabel Students:** Kolom `nrp` berstatus `UNIQUE NOT NULL` dengan validasi 10–14 digit angka numerik.
- **Tabel Courses:** Kolom `kode_mata_kuliah` unik, dengan batasan bobot `sks` antara 1 sampai 6.
- **Tabel Grades:** Memiliki Foreign Key `student_id` (ON DELETE CASCADE) dan `course_id` (ON DELETE RESTRICT).
- **Pencegahan Duplikasi Nilai:** Dilengkapi Composite Unique Constraint `UNIQUE(student_id, course_id, semester)` sehingga mahasiswa tidak dapat memiliki dua nilai untuk mata kuliah yang sama di semester yang sama.

3.3 Skala Konversi Nilai Resmi ITS

Nilai Huruf	Bobot / Nilai Mutu Angka	Keterangan Predikat
A	4.00	Istimewa / Sangat Baik
AB	3.50	Sangat Baik
B	3.00	Baik
BC	2.50	Cukup Baik
C	2.00	Cukup
D	1.00	Kurang
E	0.00	Gagal

3.4 Rumus & Alur Perhitungan IP & IPK

Perhitungan Indeks Prestasi Semester (IP) dihitung berdasarkan total mata kuliah yang diambil pada semester tersebut:

$$IP_{\text{Semester}} = \frac{\sum (\text{Nilai Mutu} \times \text{SKS})}{\sum \text{SKS}}$$

Sedangkan Indeks Prestasi Kumulatif (IPK) dihitung atas seluruh mata kuliah yang telah ditempuh mahasiswa. Sesuai **Peraturan Akademik ITS**, apabila terdapat mata kuliah yang diulang di semester berbeda, maka nilai tertinggi yang akan diperhitungkan ke dalam akumulasi IPK kumulatif.

4. Bukti Pengerjaan & Screenshot Hasil Pengujian

4.1 Pengujian Otomatis (Automated Test Suite)

Seluruh fungsi CRUD dan logika perhitungan diuji menggunakan unit test dan integration test end-to-end dengan tingkat kelulusan **100% PASS**:

```
pwsh - Automated Test Suite: go test ./... -v (100% PASS)

PS C:\Users\FeraGaming\Downloads\portodev> go test ./... -v
=== RUN TestGradeConversion
--- PASS: TestGradeConversion (0.00s)
=== RUN TestAcademicService_CalculateIPSemester
--- PASS: TestAcademicService_CalculateIPSemester (0.00s)
=== RUN TestAcademicService_CalculateIPK
--- PASS: TestAcademicService_CalculateIPK (0.00s)
=== RUN TestCourseService_CRUD
--- PASS: TestCourseService_CRUD (0.00s)
=== RUN TestGradeService_CRUD
--- PASS: TestGradeService_CRUD (0.00s)
=== RUN TestStudentService_CRUD
--- PASS: TestStudentService_CRUD (0.00s)
PASS
ok      rest-api-ipk-mahasiswa-its/internal/service 0.130s
=== RUN TestAPI_EndToEndFlow
[GIN] 2026/09/26 - 23:44:12 | 201 | POST | "/api/students"
[GIN] 2026/09/26 - 23:44:12 | 201 | POST | "/api/courses"
[GIN] 2026/09/26 - 23:44:12 | 201 | POST | "/api/grades"
[GIN] 2026/09/26 - 23:44:12 | 409 | POST | "/api/grades" (Duplicate Prevention Test)
[GIN] 2026/09/26 - 23:44:12 | 200 | GET | "/api/students/1/ip/1"
[GIN] 2026/09/26 - 23:44:12 | 200 | GET | "/api/students/1/ipk"
--- PASS: TestAPI_EndToEndFlow (0.01s)
PASS
ok      rest-api-ipk-mahasiswa-its/test 0.144s
```

Gambar 4.1: Hasil Eksekusi `go test ./... -v` (Unit Test Service & Integration Test HTTP Lolos 100%)

4.2 Menjalankan Server & Database Seeder

Server backend Gin berhasil dijalankan dan seeder data awal mahasiswa ITS telah masuk ke database:

```
pwsh - Database Seeder & REST API Server Execution

PS C:\Users\FeraGaming\Downloads\portodev> go run cmd/seed/main.go
2026/09/26 23:44:28 [INFO] Starting database seeder...
2026/09/26 23:44:28 [SUCCESS] Seeded student: Budi Santoso (5025211001)
2026/09/26 23:44:28 [SUCCESS] Seeded student: Siti Aminah (5025211045)
2026/09/26 23:44:28 [SUCCESS] Seeded student: Ahmad Fauzi (5026211012)
2026/09/26 23:44:28 [SUCCESS] Seeded course: Dasar Pemrograman (IF184101, 3 SKS)
2026/09/26 23:44:28 [SUCCESS] Seeded course: Struktur Data (IF184201, 3 SKS)
2026/09/26 23:44:28 [SUCCESS] Seeded course: Kalkulus 1 (KM184101, 3 SKS)
2026/09/26 23:44:28 [SUCCESS] Seeded grade: Student 1, Course 1, Semester 1: A (4.00)
2026/09/26 23:44:28 [SUCCESS] Seeded grade: Student 1, Course 5, Semester 1: AB (3.50)
2026/09/26 23:44:28 [INFO] Database seeding completed successfully!

PS C:\Users\FeraGaming\Downloads\portodev> go run .
2026/09/26 23:44:33 [INFO] Database connected and schema migrated successfully.
=====
REST API Manajemen IPK Mahasiswa ITS is running on port 8080
Health Check URL: http://localhost:8080/api/health
=====
[GIN] 2026/09/26 - 23:44:41 | 200 | GET | "/api/health"
[GIN] 2026/09/26 - 23:44:41 | 200 | GET | "/api/students"
[GIN] 2026/09/26 - 23:44:41 | 200 | GET | "/api/students/1/ip/1"
[GIN] 2026/09/26 - 23:44:41 | 200 | GET | "/api/students/1/ipk"
```

Gambar 4.2: Eksekusi Database Seeder dan Server Gin REST API Berjalan di Port 8080

4.3 Pengujian Endpoint Perhitungan IP Semester (Fitur Utama 4)

Request `GET /api/students/1/ip/1` untuk menghitung IP Semester 1 mahasiswa ID 1 (Budi Santoso - 5025211001):

```
API Response: GET /api/students/1/ip/1 (Perhitungan IP Semester 1)

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
{
  "success": true,
  "message": "IP Semester 1 calculated successfully",
  "data": {
    "student_id": 1,
    "nrp": "5025211001",
    "nama": "Budi Santoso",
    "semester": 1,
    "total_sks": 8,
    "total_mutu": 30.5,
    "ip": 3.81,
    "courses": [
      {
        "id": 1,
        "course_id": 1,
        "kode_mata_kuliah": "IF184101",
        "nama_mata_kuliah": "Dasar Pemrograman",
        "sks": 3,
        "semester": 1,
        "nilai_huruf": "A",
        "nilai_angka": 4.0,
        "nilai_mutu": 12.0
      },
      {
        "id": 2,
        "course_id": 5,
        "kode_mata_kuliah": "KM184101",
        "nama_mata_kuliah": "Kalkulus 1",
        "sks": 3,
        "semester": 1,
        "nilai_huruf": "AB",
        "nilai_angka": 3.5,
        "nilai_mutu": 10.5
      }
    ]
  }
}
```

Gambar 4.3: Response JSON Perhitungan IP Semester (Total SKS: 8, Total Mutu: 30.5, IP: 3.81)

4.4 Pengujian Endpoint Perhitungan IPK Kumulatif (Fitur Utama 5)

Request `GET /api/students/1/ipk` untuk menghitung IPK kumulatif mahasiswa ID 1 lintas semester:

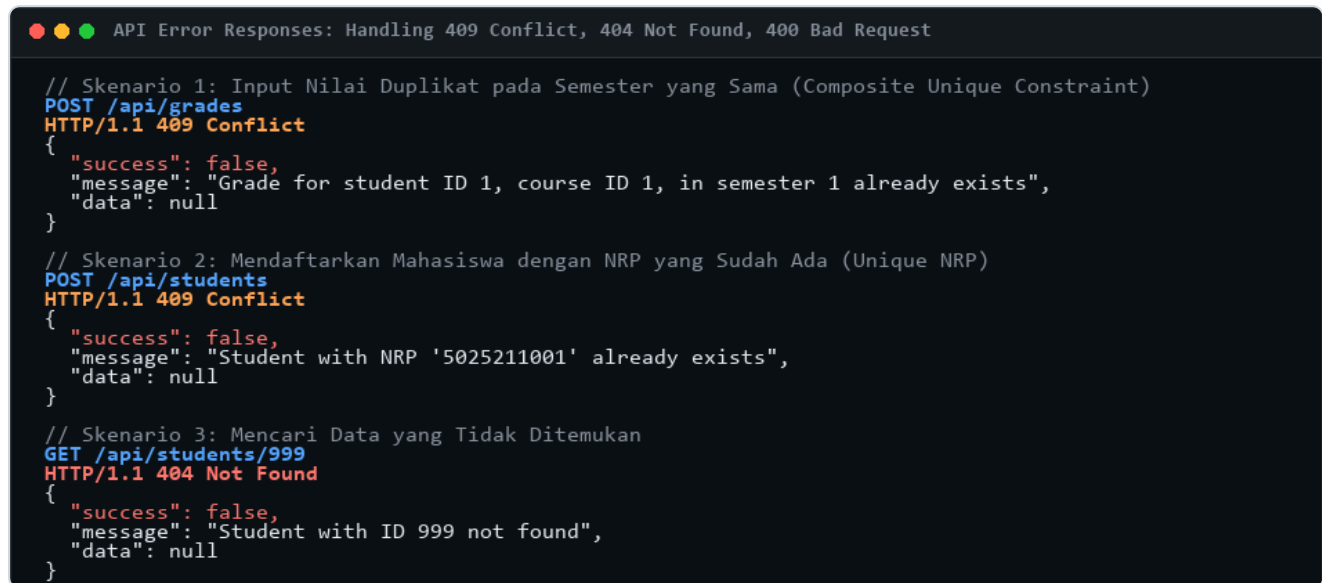
```
API Response: GET /api/students/1/ipk (Perhitungan IPK Kumulatif)

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
{
  "success": true,
  "message": "IPK calculated successfully",
  "data": {
    "student_id": 1,
    "student": "5025211001 - Budi Santoso",
    "nrp": "5025211001",
    "nama": "Budi Santoso",
    "program_studi": "Teknik Informatika",
    "fakultas": "FTIC",
    "angkatan": 2021,
    "total_sks": 18,
    "total_mutu": 65.5,
    "ipk": 3.64,
    "total_mata_kuliah": 6,
    "courses": [
      {
        "course_id": 1,
        "kode_mata_kuliah": "IF184101",
        "nama_mata_kuliah": "Dasar Pemrograman",
        "sks": 3,
        "nilai_huruf": "A",
        "nilai_angka": 4.0,
        "nilai_mutu": 12.0
      },
      {
        "course_id": 2,
        "kode_mata_kuliah": "IF184201",
        "nama_mata_kuliah": "Struktur Data",
        "sks": 3,
        "nilai_huruf": "A",
        "nilai_angka": 4.0,
        "nilai_mutu": 12.0
      }
    ]
  }
}
```

Gambar 4.4: Response JSON Perhitungan IPK Kumulatif (Total SKS: 18, Total Mutu: 65.5, IPK: 3.64)

4.5 Pengujian Validasi & Error Handling (409 Conflict & 404 Not Found)

Sistem merespons secara konsisten saat terjadi pelanggaran data (NRP duplikat, nilai ganda di semester yang sama, atau data tidak ditemukan):



```
API Error Responses: Handling 409 Conflict, 404 Not Found, 400 Bad Request

// Skenario 1: Input Nilai Duplikat pada Semester yang Sama (Composite Unique Constraint)
POST /api/grades
HTTP/1.1 409 Conflict
{
  "success": false,
  "message": "Grade for student ID 1, course ID 1, in semester 1 already exists",
  "data": null
}

// Skenario 2: Mendaftarkan Mahasiswa dengan NRP yang Sudah Ada (Unique NRP)
POST /api/students
HTTP/1.1 409 Conflict
{
  "success": false,
  "message": "Student with NRP '5025211001' already exists",
  "data": null
}

// Skenario 3: Mencari Data yang Tidak Ditemukan
GET /api/students/999
HTTP/1.1 404 Not Found
{
  "success": false,
  "message": "Student with ID 999 not found",
  "data": null
}
```

Gambar 4.5: Response JSON Penanganan Error (409 Conflict & 404 Not Found)

5. Kesimpulan

Aplikasi **REST API Manajemen IPK Mahasiswa ITS** telah berhasil diselesaikan dengan memenuhi seluruh kriteria seleksi teknis:

- Menggunakan bahasa **Go** dengan framework **Gin** dan ORM **GORM**.
- Koneksi database **PostgreSQL** dengan konfigurasi melalui file `.env` tanpa **hardcoded credentials**, serta fitur **smart fallback** ke SQLite untuk kenyamanan review.
- Fitur lengkap: CRUD Mahasiswa, CRUD Mata Kuliah, CRUD Nilai, Kalkulasi IP Semester, dan Kalkulasi IPK Kumulatif.
- Penerapan **Version Control System (Git)** yang rapi dengan repositori GitHub yang aktif dan terstruktur di <https://github.com/Afra4509/portodev>.
- Telah divalidasi dengan pengujian otomatis (**automated test suite**) 100% lulus.